

Lab 4 - Terraform + Ansible + Docker: Multi-Tier App (Flask + PostgreSQL)

Full Lab Guide, Configuration & Source Code

Terraform / Ansible / Docker / Kubernetes Lab Series

Contents

Use Case	1
Architecture	1
Directory Layout	2
Prerequisites	2
Step-by-Step Configuration & Execution	2
Step 1 — Provision the host	2
Step 2 — Install the Ansible Docker collection	3
Step 3 — Run the playbook	3
Step 4 — Verify the app	3
Validation Steps	3
Cleanup	4
Key Concepts Reinforced	4

Appendix: Full Source Code

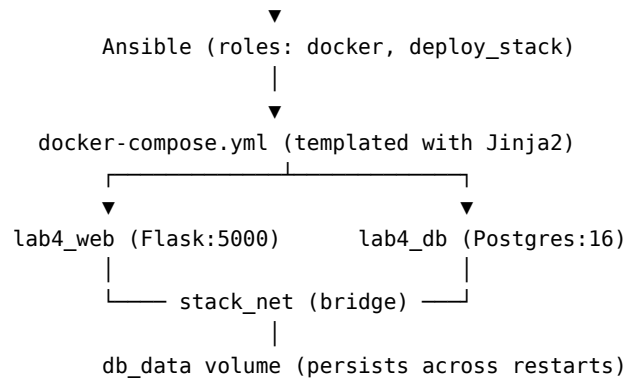
ansible/ansible.cfg	5
ansible/playbook.yml	5
ansible/requirements.yml	5
ansible/roles/deploy_stack/tasks/main.yml	5
ansible/roles/deploy_stack/templates/docker-compose.yml.j2	6
ansible/roles/docker/tasks/main.yml	7
app/Dockerfile	7
app/app.py	8
app/requirements.txt	9
terraform/main.tf	9
terraform/outputs.tf	10
terraform/variables.tf	10

Use Case

A team needs a realistic **two-tier application** — a web app (Flask) backed by a database (PostgreSQL) — deployed onto a single host as two linked containers, driven entirely by Infrastructure-as-Code. This models the most common real-world Docker Compose pattern: app + DB, private network, named volume for persistence, secrets passed as environment variables (not baked into the image).

Architecture

Terraform → EC2 instance (16GB disk) → writes ansible/inventory.ini (incl. db_password)



Directory Layout

```

lab4-tf-ansible-docker-multitier/
├── terraform/
│   ├── main.tf           # EC2 + SG (22, 5000) + inventory.ini generation w/ db_password
│   ├── variables.tf
│   └── outputs.tf
├── app/                  # Flask source (copied to remote host by Ansible)
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── ansible/
    ├── ansible.cfg
    ├── requirements.yml
    ├── playbook.yml
    └── roles/
        ├── docker/tasks/main.yml
        └── deploy_stack/
            ├── tasks/main.yml
            └── templates/docker-compose.yml.j2
  
```

Prerequisites

- Terraform >= 1.5.0, AWS CLI configured
- Ansible >= 2.14 + ansible-galaxy collection install -r ansible/requirements.yml
- EC2 key pair + local .pem

Step-by-Step Configuration & Execution

Step 1 — Provision the host

```

cd terraform
terraform init
terraform apply \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="db_password=SuperSecretPass123!"
  
```

This writes ../ansible/inventory.ini including db_password as an inventory var, which the Compose template consumes — the secret never lives in a Dockerfile or is committed to the compose file in git.

Step 2 — Install the Ansible Docker collection

```
cd ../ansible
ansible-galaxy collection install -r requirements.yml
```

Step 3 — Run the playbook

```
until ansible stack_hosts -m ping; do sleep 5; done
ansible-playbook playbook.yml
```

This installs Docker + Compose plugin, copies the Flask app source to /opt/stack/app, templates docker-compose.yml with the real DB password, and runs `docker compose up --build -d` via the `community.docker.docker_compose_v2` module (idempotent: re-running only rebuilds/restarts what changed).

Step 4 — Verify the app

```
curl http://$(cd ../terraform && terraform output -raw stack_host_public_ip):5000/
```

Refresh a few times — the visit counter should increment, proving the Flask container is actually persisting state in the Postgres container.

Validation Steps

1. Both containers running

```
ansible stack_hosts -m shell -a "docker ps --format
'{{ '{{' }}.Names{{ '{{' }}}\t{{ '{{' }}.Status{{ '{{' }}}}'"
# Expect: lab4_web (Up), lab4_db (Up)
```

2. Network isolation — DB is not exposed publicly

```
nc -zv -w3 <public_ip> 5432 # should FAIL (connection refused/timeout)
nc -zv -w3 <public_ip> 5000 # should SUCCEED
```

3. App ↔ DB connectivity

```
curl -s http://<public_ip>:5000/health # {"status": "ok"}
curl -s http://<public_ip>:5000/ | grep "Total visits"
```

4. Data persistence across container restart

```
ansible stack_hosts -m shell -a "cd /opt/stack && docker compose restart"
curl -s http://<public_ip>:5000/ | grep "Total visits" # counter continues, doesn't reset to 0
```

5. Named volume actually holds the data

```
ansible stack_hosts -m shell -a "docker volume inspect stack_db_data"
```

6. Idempotent re-apply

```
ansible-playbook playbook.yml # second run should show "changed=0" for docker install tasks
```

Cleanup

```
cd terraform
terraform destroy \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="db_password=SuperSecretPass123!"
```

Key Concepts Reinforced

- Passing secrets from Terraform → Ansible inventory → Compose template (never hardcoded)
- Docker Compose networking: only intentionally-published ports are reachable externally
- Named volumes for stateful services
- `community.docker.docker_compose_v2` for idempotent multi-container deployment

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/ansible.cfg

```
[defaults]
inventory = inventory.ini
host_key_checking = False
retry_files_enabled = False
```

ansible/playbook.yml

```
---
- name: Configure host with Docker + Compose, deploy multi-tier stack
  hosts: stack_hosts
  become: true
  gather_facts: true

  roles:
    - docker
    - deploy_stack
```

ansible/requirements.yml

```
---
collections:
  - name: community.docker
    version: ">=3.4.0"
```

ansible/roles/deploy_stack/tasks/main.yml

```
---
- name: Create stack directory
  file:
    path: /opt/stack
    state: directory
    mode: "0755"

- name: Create app subdirectory
  file:
    path: /opt/stack/app
    state: directory
    mode: "0755"

- name: Copy Flask app source
  copy:
    src: "{{ playbook_dir }}/../../app/{{ item }}"
    dest: /opt/stack/app/{{ item }}
    mode: "0644"
```

```
loop:
  - app.py
  - requirements.txt
  - Dockerfile

- name: Template docker-compose.yml with the DB password
  template:
    src: docker-compose.yml.j2
    dest: /opt/stack/docker-compose.yml
    mode: "0644"

- name: Bring up the stack with Docker Compose
  community.docker.docker_compose_v2:
    project_src: /opt/stack
    state: present
    build: always
```

ansible/roles/deploy_stack/templates/docker-compose.yml.j2

```
version: "3.9"

services:
  web:
    build: ./app
    container_name: lab4_web
    restart: always
    environment:
      DB_HOST: db
      DB_NAME: appdb
      DB_USER: appuser
      DB_PASSWORD: "{{ db_password }}"
    ports:
      - "5000:5000"
    depends_on:
      - db
    networks:
      - stack_net

  db:
    image: postgres:16-alpine
    container_name: lab4_db
    restart: always
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: "{{ db_password }}"
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - stack_net

volumes:
  db_data:
```

```
networks:
  stack_net:
```

ansible/roles/docker/tasks/main.yml

```
---
- name: Install prerequisites
  apt:
    name: [apt-transport-https, ca-certificates, curl, gnupg, lsb-release]
    state: present
    update_cache: true

- name: Add Docker GPG key
  apt_key:
    url: https://download.docker.com/linux/ubuntu/gpg
    state: present

- name: Add Docker apt repository
  apt_repository:
    repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_distribution_release }} stable"
    state: present

- name: Install Docker Engine + Compose plugin
  apt:
    name: [docker-ce, docker-ce-cli, containerd.io, docker-compose-plugin]
    state: present
    update_cache: true

- name: Start and enable Docker
  systemd:
    name: docker
    state: started
    enabled: true

- name: Add ubuntu user to docker group
  user:
    name: ubuntu
    groups: docker
    append: true
```

app/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

app/app.py

```
import os
import time
import psycopg2
from flask import Flask

app = Flask(__name__)

DB_HOST = os.environ.get("DB_HOST", "db")
DB_NAME = os.environ.get("DB_NAME", "appdb")
DB_USER = os.environ.get("DB_USER", "appuser")
DB_PASSWORD = os.environ.get("DB_PASSWORD", "changeme")

def get_conn(retries=10, delay=3):
    last_err = None
    for _ in range(retries):
        try:
            return psycopg2.connect(
                host=DB_HOST, dbname=DB_NAME, user=DB_USER, password=DB_PASSWORD
            )
        except Exception as e: # noqa: BLE001
            last_err = e
            time.sleep(delay)
    raise last_err

def init_db():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute(
        "CREATE TABLE IF NOT EXISTS visits (id SERIAL PRIMARY KEY, ts TIMESTAMP DEFAULT NOW())"
    )
    conn.commit()
    cur.close()
    conn.close()

@app.route("/")
def index():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("INSERT INTO visits DEFAULT VALUES")
    conn.commit()
    cur.execute("SELECT COUNT(*) FROM visits")
    count = cur.fetchone()[0]
    cur.close()
    conn.close()
    return f"<h1>Multi-tier Lab 4</h1><p>Total visits recorded in Postgres: {count}</p>"

@app.route("/health")
def health():
    return {"status": "ok"}
```

```
if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000)
```

app/requirements.txt

```
flask==3.0.3
psycpg2-binary==2.9.9
```

terraform/main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name     = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name     = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "stack" {
  name_prefix = "lab4-stack-sg-"
  vpc_id      = data.aws_vpc.default.id
}
```

```

ingress { description = "SSH"      from_port = 22  to_port = 22  protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
ingress { description = "Web App"  from_port = 5000 to_port = 5000 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
egress  { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

tags = { Name = "lab4-stack-sg" }
}

```

```

resource "aws_instance" "stack_host" {
  ami              = data.aws_ami.ubuntu.id
  instance_type    = var.instance_type
  subnet_id        = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.stack.id]
  key_name          = var.key_name
  root_block_device {
    volume_size = 16
  }
}

```

```

tags = { Name = "lab4-multitier-host" }
}

```

```

resource "local_file" "ansible_inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
  content  = <<-EOT
[stack_hosts]
${aws_instance.stack_host.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansi
o StrictHostKeyChecking=no'

[stack_hosts:vars]
db_password=${var.db_password}
EOT
}

```

terraform/outputs.tf

```

output "stack_host_public_ip" {
  value = aws_instance.stack_host.public_ip
}

```

terraform/variables.tf

```

variable "aws_region"      { default = "us-east-1" }
variable "instance_type"   { default = "t3.small" }
variable "key_name"        { type = string }
variable "private_key_path" { type = string }
variable "db_password" {
  type      = string
  sensitive = true
}

```